AMRITA VISHWA VIDYAPEETHAM

श्रद्धावान् लभते ज्ञानम्

NAME: RAAGHAV VEL P

CSE-B

CH.SC.U4CSE24155

1) Sum of Natural Numbers:

CODE:

```
#include <stdio.h>

int main(){
  int n;
  printf("Enter number:");
  scanf("%d",&n);
  int sum=0;
  for(int i=1;i<n+1;i++){
    sum=sum+i;
  }
  printf("sum is %d\n",sum);
  return 0;
}
```

OUTPUT:

```
Enter number:10
sum is 55
```

JUSTIFICATION:

Space Complexity: O(1)

The sum of natural numbers is calculated using a loop and a few integer variables. The program's memory usage is independent of the input size because it does not store intermediate results in any variable. Therefore, the space complexity remains constant.

2) Sum of Squares

CODE:

```
#include <stdio.h>

int main(){
  int n;
  printf("Enter number:");
  scanf("%d",&n);
  int sum=0;
  for(int i=1;i<n+1;i++){
    sum=sum+i*i;
  }
  printf("sum is %d\n",sum);
  return 0;
}
```

OUTPUT:

```
Enter number:10
sum is 385
```

JUSTIFICATION:

Space Complexity: O(1)

This program computes the sum of squares using a loop and a small, fixed number of variables. the amount of memory used remains unchanged for all input sizes, resulting in constant space complexity.

3) Sum of Cubes Program

CODE:

```
#include <stdio.h>

int main(){
  int n;
  printf("Enter number:");
  scanf("%d",&n);
  int sum=0;
  for(int i=1;i<n+1;i++){
    sum=sum+i*i*i;
  }
  printf("sum is %d\n",sum);
  return 0;
}
```

OUTPUT:

```
Enter number:10
sum is 3025
```

JUSTIJICATION:

Space Complexity: O(1)

The program calculates the sum of cubes using a loop and maintains only a fixed set of variables for computation. The memory requirement does not increase with the input value, as no additional storage is created during execution. Hence, the space complexity is constant.

4) Factorial (Recursive)

CODE:

```
#include <stdio.h>

int fac(int a){
  if(a==1){
    return 1;
  }
  else{
    return a*fac(a-1);
  }
}

int main(){
  int n;
  printf("Enter number:");
  scanf("%d",&n);
  printf("factorial is %d\n",fac(n));
  return 0;
}
```

OUTPUT:

```
Enter number:5
factorial is 120
```

JUSTIFICATION:

Space Complexity: O(n)

program uses recursion, and each recursive call requires its own stack frame to store the function parameter and return information. As the input value increases, the number of recursive calls increases proportionally.

5) Matrix Transpose

CODE:

```
#include <stdio.h>

int main(){
  int n[3][3]={{1,2,3},{4,5,6},{7,8,9}};
  int transpo[3][3];
  for(int i=0;i<3;i++){
    for(int j=0;j<3;j++){
      transpo[i][j]=n[j][i];
      printf("%d",transpo[i][j]);
    }
    printf("\n");
  }

}
```

OUTPUT:

```
147
258
369
```

JUSTIFICATION:

Space Complexity: O(1)

the amount of memory required remains the same for every execution. No additional dynamic memory or data structures are used, so the space required is constant.

6) Fibonacci Series (Iterative)

CODE:

```
#include <stdio.h>

int main(){
  int n;
  printf("Enter number:");
  scanf("%d",&n);
  int f0=0,f1=1;
  printf("\n%d %d ",f0,f1);
  for(int i=1;i<n+1;i++){
    int temp=f1;
    f1=f0+f1;
    f0=temp;
    printf("%d ",f1);
  }
  return 0;
}
```

OUTPUT:

```
C:\II year\4th sem\DAA\LAB\week-1\prog>a
Enter number:10

0 1 1 2 3 5 8 13 21 34 55 89
```

JUSTIFICATION:

Space Complexity: O(1)

stores only a fixed number of integer variables to keep track of the previous and current values. the program does not allocate extra memory that grows with the input, so the space usage remains constant.